## Assignment 04 CODE -

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scannerscanner=newScanner(System.in);
        int INF = 999;
        int n;

        System.out.print("Enternumberofnodes: ");
        n = scanner.nextInt();

        int[][] cost = new int[20][20];
        int[][] dist = new int[20][20];
        int[][] nextHop = new int[20][20];

        // Input cost matrix[cite: 1]
        System.out.println("\nEnterthecostmatrix:");
        System.out.println("Enter999forinfinity(no direct connection).\n");

        for (int i = 0; i < n; i++) {
            System.out.print("Row"+(i+1)+":");
            for (int j = 0; j < n; j++) {
                cost[i][j] = scanner.nextInt();

                //Initialdistanceisthesameascost[cite: 1]
                dist[i][j] = cost[i][j];

                // Set next hop[cite: 1]
                if (i == j) {
                    nextHop[i][j] = -1;
                } else if (cost[i][j] != INF) {
                    nextHop[i][j] = j;
                } else {
                    nextHop[i][j] = -1;
                }
            }
        }

        // Display initial routing tables[cite: 2]
        System.out.println("\n==========INITIAL ROUTING TABLES ==========");

        for (int i = 0; i < n; i++) {
            System.out.println("\nRouter"+(i+1));
            System.out.println("Destination\tCost\tNext Hop");
```

```
    for (int j = 0; j < n; j++) {
        System.out.print((j + 1) + "\t\t");

        if (dist[i][j] == INF) {
            System.out.print("INF\t");
        } else {
            System.out.print(dist[i][j] + "\t");
        }

        if (nextHop[i][j] == -1) {
            System.out.println("-");
        } else {
            System.out.println((nextHop[i][j] + 1));
        }
    }
}

// Distance Vector Algorithm[cite: 2]
boolean changed;

do{
    changed = false;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < n; k++) {
                //Checkificanreachkandkcanreach j[cite: 2]
                if (cost[i][k] != INF && dist[k][j] != INF) {
                int newCost = cost[i][k] + dist[k][j];
                if (newCost < dist[i][j]) {
                        dist[i][j] = newCost;
                        nextHop[i][j] = nextHop[i][k];
                        changed = true;
                    }
                }
            }
        }
    }
} while (changed);

// Display final routing tables[cite: 3]
System.out.println("\n==========FINALROUTING TABLES ==========");

for (int i = 0; i < n; i++) {
    System.out.println("\nRouter " + (i + 1));
    System.out.println("Destination\tCost\tNextHop");

    for (int j = 0; j < n; j++) {
        System.out.print((j + 1) + "\t\t");
```

```
            if(dist[i][j]== INF) {
                System.out.print("INF\t");
            } else {
                System.out.print(dist[i][j] + "\t");
            }

            if(nextHop[i][j] == -1) {
                System.out.println("-");
            } else {
                System.out.println((nextHop[i][j] + 1));
            }
        }
    }

    //Findshortestpath[cite: 3]
    intsource,destination;

    System.out.print("\nEnter source node: ");
    source=scanner.nextInt();

    System.out.print("Enter destination node: ");
    destination=scanner.nextInt();

    source--;
    destination--;

    if(dist[source][destination] == INF) {
        System.out.println("\nNo path exists between the selected nodes.");
    } else {
        System.out.println("\n========== SHORTEST PATH ==========");
        System.out.print("Path: ");
        intcurrent=source;
        System.out.print(current + 1);

        while(current!= destination) {
            current=nextHop[current][destination];
            System.out.print(" -> " + (current + 1));
        }

        System.out.println("\nMinimum Cost: " + dist[source][destination]);
    }

    scanner.close();
  }
}
```

**OUTPUT -**

```
Enter number of nodes: 4

Enter the cost matrix:
Enter 999 for infinity (no direct connection).

Row 1: 0 2 5 10
Row 2: 2 0 999 1
Row 3: 5 999 0 2
Row 4: 10 1 2 0

========== INITIAL ROUTING TABLES ==========

Router 1
Destination Cost    Next Hop
1       0   -
2       2   2
3       5   3
4       10  4

Router 2
Destination Cost    Next Hop
1       2   1
2       0   -
3       INF -
4       1   4

Router 3
Destination Cost    Next Hop
1       5   1
2       INF -
3       0   -
4       2   4

Router 4
Destination Cost    Next Hop
1       10  1
2       1   2
```

**GITHUB LINK-**

**https://github.com/akkishansh/computer-networks-lab/**